IQRASEC ACADEMY

Burp Suite Repeater Report

WAPT Phase 1

Task 3 Burp Suite

Prepared by

Kinza Noor

Report date

3 October 2026

Learning platform

TryHackMe

Description:

This report explains a repeatable approach to editing HTTP requests and evaluating responses in Burp Suite Repeater. It covers the room exercises and provides editable fields for actual observations and screenshots. Practical results and completion must be supported by your own lab evidence.

Table of Contents

1 Objective 3
2 Concept and Understanding 3
2.1 Repeater and Proxy 3
2.2 Request Editing and History 3
2.3 Message Views and Inspector 3
2.4 Interpreting Test Responses 3
3 Practical Work 3
3.1 Environment and Baseline 3
3.2 Message Views and Inspector Practice 4
3.3 Header Modification Exercise 4
3.4 Product Input Challenge 4
3.5 SQL Injection Challenge 5
4 Tools Used 6
5 Evidence 6
6 Result 7
7 Key Learnings 7
8 References 8

1 Objective

To capture a baseline request, modify one input at a time and compare the resulting response. The exercises develop manual testing skills for headers, path parameters, error handling and an assigned SQL injection challenge.

2 Concept and Understanding

2.1 Repeater and Proxy

Proxy captures browser traffic. Repeater holds a request for manual editing and repeated transmission without requiring another browser action. Each response belongs to a particular sent request, so keeping a baseline makes later changes easier to explain.

2.2 Request Editing and History

Requests can be transferred using Send to Repeater or Ctrl+R. Check the destination host, port and transport before sending. Use separate named tabs for different tests and history controls to revisit previous request and response pairs. A request in the editor has no test result until it is sent.

2.3 Message Views and Inspector

Pretty supports readable message inspection; Raw shows message text, Hex supports byte-level review and Render presents supported response content visually. Inspector provides structured access to request attributes, parameters, cookies and headers. Editable request fields change the outgoing message; response headers describe what the server returned.

Body parameters are not inherently exclusive to POST: other methods can carry bodies too. Available Inspector sections depend on the message and its content type. Interface labels may differ between Burp versions.

2.4 Interpreting Test Responses

Compare status, headers, body content and response size with the baseline. A 500 error establishes a server-side failure, but does not by itself prove SQL injection. Stronger evidence links controlled input changes to database behavior or retrieved lab data. Repeat a test when needed to establish that the result is reproducible.

3 Practical Work

3.1 Environment and Baseline

Deploy the room machine and open Burp on the AttackBox or VPN-connected Kali. Use Burp's browser, or configure your browser to use the active Proxy listener. Replace 10.49.166.80 with the displayed lab IP. Turn interception off during ordinary browsing to avoid leaving requests waiting.

Target IP: [Enter IP]
Burp version and edition: [Enter details]
Environment and practice date: [Enter details]

http:// 10.49.166.80/

Find the home-page request in Proxy HTTP history, send it to Repeater and press Send. Save a screenshot of the request and baseline response. Use a descriptive tab name such as Home baseline.

3.2 Message Views and Inspector Practice

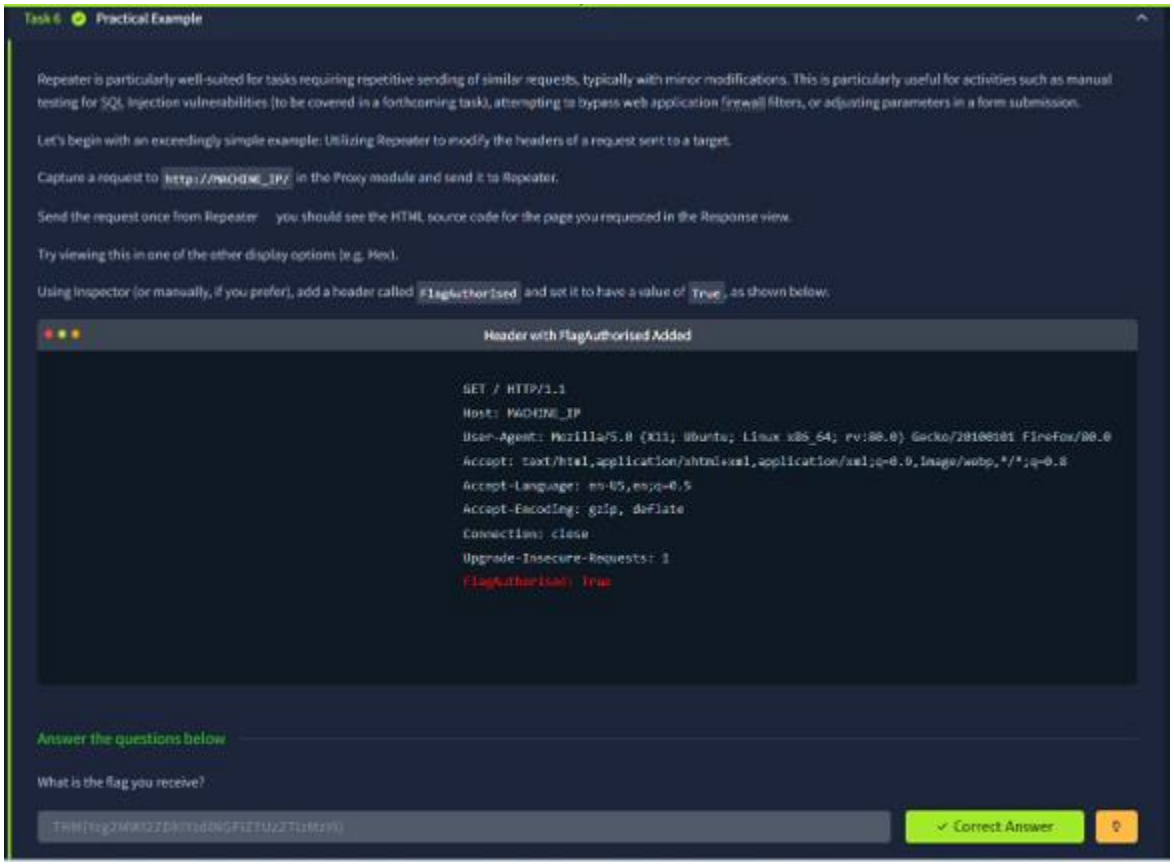
Inspect the same response in Pretty, Raw, Hex and Render where available. Use Inspector to review request headers. Make a small request-header change, send it and compare the response with the baseline. Use history to return to the original request.

3.3 Header Modification Exercise

For the home-page exercise, add the following header to the captured request using Inspector or the message editor. Keep an empty line between the headers and any body, then send the request.

FlagAuthorised: True

Capture the modified request beside the returned flag. Record the actual response rather than assuming the header was accepted. A client-supplied authorization marker is under the client's control; secure authorization must be enforced by the server.

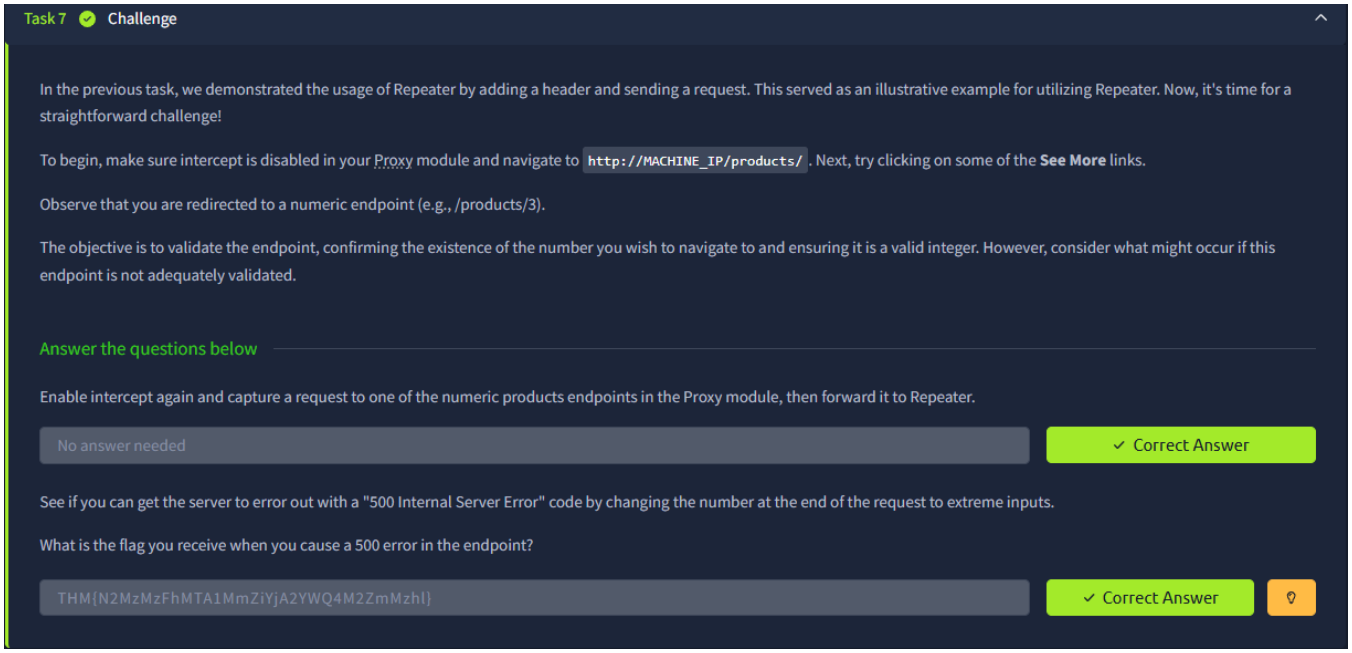


3.4 Product Input Challenge

http:// 10.49.166.80/products/
http:// 10.49.166.80/products/3

Capture a numeric product request and establish a normal response. Change the final path segment to an extreme or invalid value, such as a very large integer or nonnumeric text. Record each exact input and outcome; stop once the assigned error behavior is demonstrated.

Get Error 500: /products/-84



3.5 SQL Injection Challenge

http:// 10.49.166.80/about/2

Capture this request and preserve the successful baseline. The assigned challenge uses the ID segment of /about/ID. Append an apostrophe and compare the response. Record any query details disclosed in the error body.

/about/2'

The room walkthrough identifies a five-expression query against people. Confirm this from your lab response. UNION queries require matching column counts and compatible types. NULL placeholders help preserve the shape of the result while one expression supplies visible data.

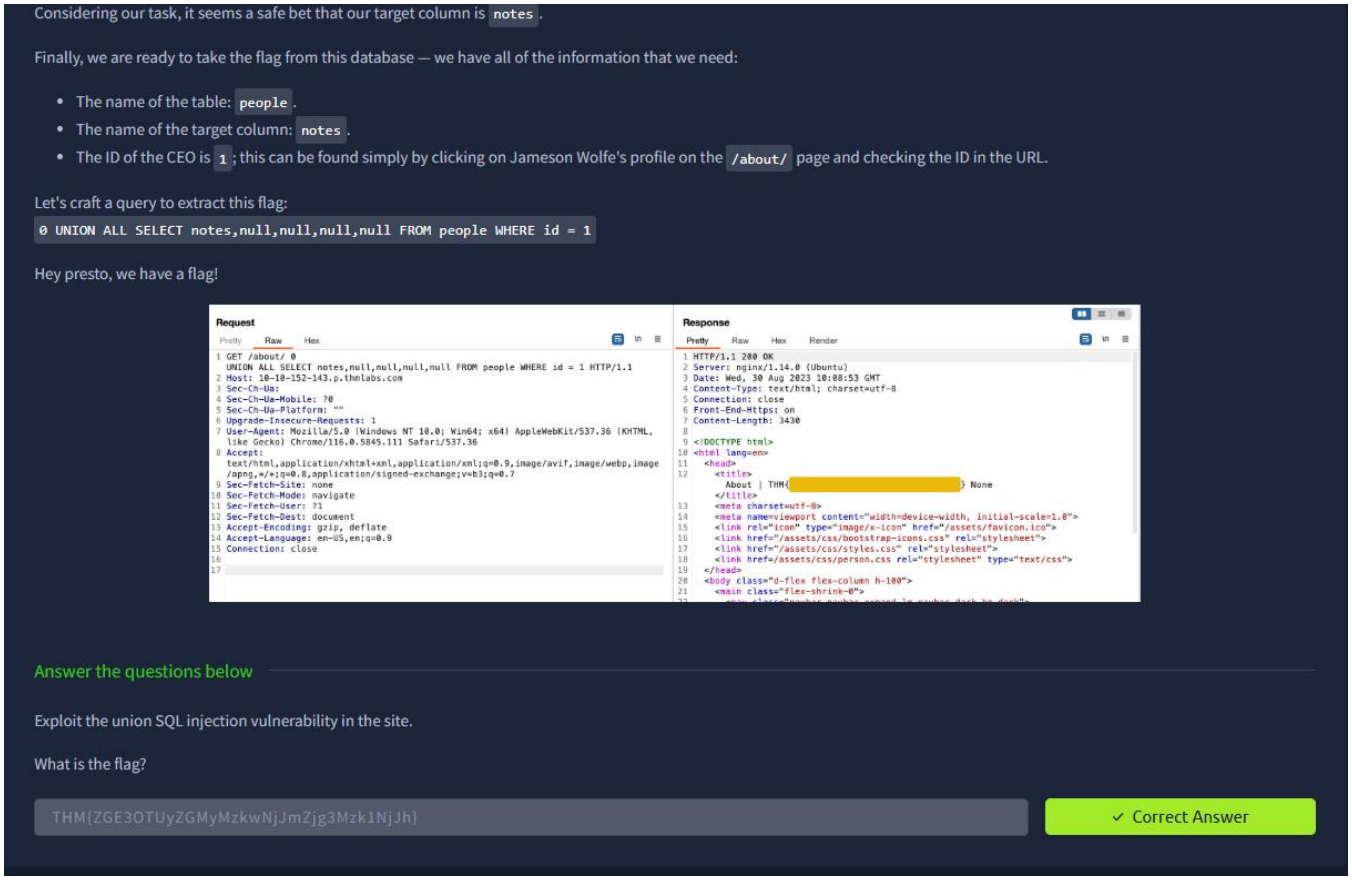
Use the following lab payload to inspect column metadata. These are decoded path values: encode spaces as %20 and double quotes as %22 before sending them in an HTTP request line. Inspector can help edit and encode the path.

```
/about/0 UNION ALL SELECT
 group_concat(column_name),NULL,NULL,NULL,NULL
 FROM information_schema.columns WHERE table_name="people"
```

The line breaks above are for readability; enter the payload as a single path. Review the returned column names. Aggregation combines metadata into one value, which is useful when the page displays only one result. Verify the relevant profile ID before selecting its notes.

```
/about/0 UNION ALL SELECT notes,NULL,NULL,NULL,NULL
 FROM people WHERE id=1
```

This final payload targets the CEO notes in the assigned lab. It is a test input, not a captured result. Preserve the returned data and exact sent request as evidence. The zero ID is intended to suppress the ordinary row; validate its behavior against the current application.



4 Tools Used

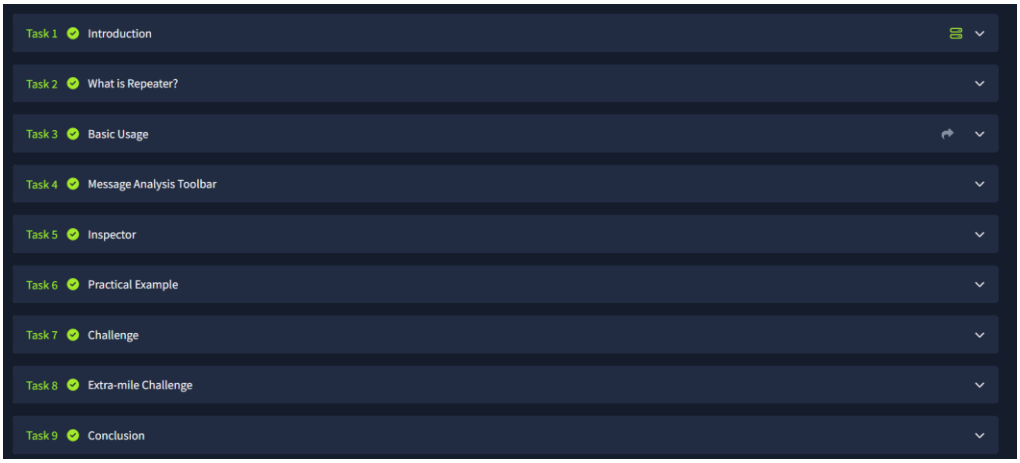
Tool	Purpose
Burp Proxy and browser	Capture the lab application requests.
Burp Repeater	Edit and resend individual requests.
Inspector and message views	Review fields, encoding and response content.
TryHackMe lab machine	Provide the assigned training application.

5 Evidence

Insert screenshots with the request and matching response readable. Keep the target and changed input visible. Add the caption beneath each image and refer to its figure number in your findings.

Figure	Screenshot to insert
1	Repeater home-page baseline request and response.
2	Inspector or message-view practice with the relevant field visible.
3	FlagAuthorised: True header and returned challenge flag.

Figure	Screenshot to insert
4	Changed product identifier and its error response or flag.
5	SQL test input with disclosed query or column metadata.
6	Final UNION request and returned CEO notes or flag.
7	Room title and your completion progress.



6 Result



7 Key Learnings

Repeater supports controlled manual tests by preserving and resending requests. Reliable conclusions come from comparing a normal response with a specific modification. Inspector simplifies editing, while raw messages show what is sent. Error disclosures can assist diagnosis, but a server error alone is

insufficient to establish the cause. Parameterized SQL and server-side validation address the underlying weaknesses illustrated by the exercises.

8 References

TryHackMe. Burp Suite Repeater. https://tryhackme.com/room/burpsuiterepeater

PortSwigger. Burp Repeater. https://portswigger.net/burp/documentation/desktop/tools/repeater

PortSwigger. Inspector. https://portswigger.net/burp/documentation/desktop/tools/inspector

PortSwigger Web Security Academy. SQL injection UNION attacks. https://portswigger.net/web-security/sql-injection/union-attacks